

Prompt Optimization for Large Language Models via Squirrel Search Algorithm

Yash Patel
Stetson University
DeLand, Florida, USA
ypatel2@stetson.edu

Nour Amri
Stetson University
DeLand, Florida, USA
nouramri612@gmail.com

Hala ElAarag
Stetson University
DeLand, Florida, USA
helaarag@stetson.edu

Abstract

Adapting Large Language Models (LLMs) to perform effectively on specific tasks remains challenging. Traditional methods such as Full Fine-Tuning (FFT) employ backpropagation to update model parameters, incurring substantial computational and temporal costs. Furthermore, access to internal parameters is restricted for black-box models deployed through APIs, rendering gradient-based approaches infeasible. Parameter-Efficient Fine-Tuning (PEFT) techniques, particularly prompt tuning, address these limitations by optimizing task performance on frozen LLMs through systematic prompt engineering rather than parameter modification. However, prompt optimization constitutes a discrete combinatorial problem amenable to metaheuristic solutions that seek near-optimal configurations rather than guaranteed global optima. This work presents a novel application of the Squirrel Search Algorithm (SSA) to automated prompt optimization, encoding prompts as continuous genome vectors representing instruction templates, reasoning strategies, and output formats. We evaluate our approach on sentiment analysis tasks using 3.8B-20B parameter models, achieving relative fitness reductions of up to 57% with convergence within 10-33 iterations. Our results demonstrate that nature-inspired metaheuristic algorithms provide a viable gradient-free alternative for black-box LLM adaptation.

CCS Concepts

- **Computing methodologies** → **Natural language processing**;
- **Mathematics of computing** → **Evolutionary algorithms**.

Keywords

large language models, prompt tuning, evolutionary optimization, squirrel search algorithm, metaheuristics

ACM Reference Format:

Yash Patel, Nour Amri, and Hala ElAarag. 2026. Prompt Optimization for Large Language Models via Squirrel Search Algorithm. In *2026 ACM Southeast Conference (ACMSE 2026)*, April 23–25, 2026, Troy, AL, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3746467.3801519>



This work is licensed under a Creative Commons Attribution 4.0 International License. *ACMSE 2026, April 23–25, 2026, Troy, AL, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2062-8/2026/04
<https://doi.org/10.1145/3746467.3801519>

1 Introduction

Large Language Models (LLMs) demonstrate emergent reasoning abilities across diverse domains; however, achieving optimal performance for task-specialized inferencing remains a key challenge. Historically, full fine-tuning (FFT) has been the most common strategy for adaptation [5], where parameters are retrained using additional supervision after pretraining. While effective, FFT introduces prohibitive computational cost and memory requirements, particularly as model sizes scale to billions of parameters [20]. Moreover, FFT is inaccessible for commercial black-box LLMs that prevent gradient updates and internal parameter access.

This has motivated alternative adaptation strategies collectively known as Parameter-Efficient Fine-Tuning (PEFT) [20], where instead of updating all model weights, only peripheral structures or inputs are optimized. Among these approaches, prompt tuning—optimizing natural language instructions fed into frozen models—has emerged as a lightweight but powerful alternative. Yet prompt engineering is highly sensitive [15]: lexical changes can dramatically alter model reasoning, and the combinatorial space of possible prompts creates irregular optimization surfaces unsuitable for gradient descent. Thus, prompt optimization lends itself to stochastic search methods where population-based metaheuristics can explore large solution spaces without requiring gradients [4].

In this work, we introduce a nature-inspired metaheuristic, Squirrel Search Algorithm (SSA)—for automated prompt optimization. We encode prompts as continuous-valued genomes which represent instruction templates, output constraints, reasoning strategies, and formatting specifications. SSA iteratively mutates these genomes and evaluates fitness based on classification performance. We demonstrate that SSA substantially improves baseline prompts for multiple LLMs and resulting in structured prompt refinements such as role specialization and simplified output formatting. Source code is available on GitHub¹.

2 Related Work

2.1 Gradient-Free Prompt Optimization

Gradient-free prompt tuning methods have grown in popularity due to their compatibility with closed-weight models. One seminal method, GriPS [15], performs local, edit-based mutations over natural language instructions using operations such as paraphrasing, deletion, and token swapping. It evaluates modified prompts and retains higher-scoring variants through greedy selection.

¹<https://github.com/YashPatel04/SSA-Optimizer>

In contrast, population-based metaheuristics maintain diversity and search broader regions [1]. Our implementation of SSA employs structured population-based behavioral phases to balance convergence speed with the capacity to escape local minima.

2.2 Evolutionary Prompting

Recent works have explored population-based metaheuristics to maintain search diversity [2]. A notable example is EvoPrompt [4], which connects LLMs with evolutionary algorithms such as genetic algorithms (GA) [17] and differential evolution (DE) [16]. In EvoPrompt, the LLM itself serves as the evolutionary operator, performing crossover and mutation directly on discrete natural language strings to ensure the generated instructions remain coherent and human-readable.

While our work shares the goal of maintaining search diversity through population-based methods, we depart from EvoPrompt’s discrete string manipulation. Instead, we propose a structured encoding of prompt components including instruction templates, reasoning strategies, and output formats into a continuous 10-dimensional genome, which allows SSA to leverage structured behavioral phases to navigate the irregular optimization landscape of LLM prompts.

2.3 Squirrel Search Algorithm (SSA)

Metaheuristic algorithms such as GA [17], particle swarm optimization (PSO) [10], and DE [16] have proven effective for NLP and text-based tasks involving high dimensionality, nonlinearity, and dynamic search spaces [6].

SSA, introduced by Jain et al. [8], models the forage, gliding, seasonal migration, and risk-driven behavior of southern flying squirrels. Population agents adjust positions relative to elite acorns (food sources) based on stochastic movement rules. SSA has demonstrated performance in engineering optimization, machine learning, and scheduling problems due to its simplicity, fast convergence, and strong balance between exploration and exploitation [3].

Positions in SSA correspond to candidate solutions—in our case, encoded prompts. The algorithm dynamically migrates poorer agents toward elites while introducing Lévy noise to prevent stagnation. This stochastic, gradient-free search makes SSA well-suited for discrete, black-box optimization domains such as prompt engineering.

3 Problem Definition

The task-specific adaptation of LLMs through prompt engineering has become a central yet difficult challenge in contemporary NLP. Prompt optimization faces several core obstacles: prompt brittleness, in which small lexical changes yield large performance variations; limited embedded domain knowledge; and constraints in models’ instruction-following abilities [11]. Manual prompt design demands extensive trial-and-error, making it resource-intensive and underscoring the need for automated prompt-optimization algorithms.

Prompt optimization is fundamentally a combinatorial problem. With a vocabulary of size $|V|$ and prompt length L , the search space includes $|V|^L$ configurations, and typically $|V| \gg L$. This exponential growth renders exhaustive search infeasible [18]. Effective

methods must therefore navigate local optima while identifying globally competitive solutions.

A key practical constraint is the black-box paradigm governing most state-of-the-art LLM deployments. Commercial API-based models such as GPT, Gemini, and Claude expose only inference interfaces, denying access to parameters or gradients. As a result, gradient-based optimization is impossible. This limitation removes an entire class of techniques relying on backpropagation and introduces strict query-efficiency demands imposed by rate limits and per-query costs.

To address these constraints, this work develops a gradient-free prompt optimization framework. We employ SSA to explore discrete prompt space via a continuous genome encoding scheme.

4 Methodology

We develop a gradient-free prompt tuning framework where prompt structures are encoded as continuous genome vectors, enabling population-based optimization using the Squirrel Search Algorithm (SSA). The 10-dimensional genome balances expressiveness with search tractability, while SSA hyperparameters ($G_c = 1.9$, $P_{dp} = 0.1$) are adopted from Jain et al. [8] as implementation choices rather than algorithmic requirements.

4.1 Prompt Representation and Genome Encoding

To parameterize prompt search, we encode a prompt as a 10-dimensional vector

$$\mathbf{g} = [g_1, g_2, \dots, g_{10}]$$

where each g_i controls a semantic or structural component of a generated instruction.

The genome decodes into instruction templates, reasoning strategy, formatting style, lexical constraints, role personas, and few-shot examples, as illustrated in Table 1. Each dimension is interpreted through deterministic mapping thresholds or categorical lookup, enabling SSA to search continuous space while mutating discrete prompt components.

Table 1: Genome Structure and Component Mapping

Index	Component	Encoding
g_1	Instruction template ID	Discrete: 0–15
g_2	Reasoning flag	Binary: {0, 1}
g_3	Reasoning template ID	Discrete: 0–12
g_4	Output format ID	Discrete: 0–14
g_5	Constraint strength	Continuous: [0,1]
g_6	Role assignment flag	Binary: {0, 1}
g_7	Role template ID	Discrete: 0–11
g_8	Synonym intensity	Continuous: [0,1]
g_9	Example flag	Binary: {0, 1}
g_{10}	Example count	Discrete: 0–8

Mutations apply synonym substitution, formatting switches, role changes, reasoning modes, and few-shot demonstrations. A sample genome for illustration is:

$$[4.84, 1.78, 1.59, 0.66, 0.23, 0.85, 3.26, 0.86, 0.01, 3.06]$$

4.2 Squirrel Search Algorithm (SSA)

We adapt SSA (Algorithm 1) for prompt optimization by treating each genome vector as a squirrel position in search space. Population dynamics are stratified based on fitness rank:

- **15% elite** – exploitation toward current best
- **35% exploratory** – Lévy flight stochastic search
- **50% foraging** – predator disturbance and random large jumps

Let p denote population size, T maximum iterations, $d=10$ genome dimensions, and $j \in \{1, \dots, d\}$ index individual dimensions.

4.2.1 Elite Movement. Elite squirrels exploit the current best solution via a linearly decaying weight:

$$W = 2 \left(1 - \frac{t}{T} \right) \quad (1)$$

$$g_j \leftarrow g_j + G_c \cdot W \cdot (g_{\text{best},j} - g_j) \cdot r_j, \quad r_j \sim U[0, 1] \quad (2)$$

where $G_c = 1.9$ is the gliding constant and t is the current iteration.

4.2.2 Exploratory Movement (Lévy Flight). Exploratory squirrels perform Lévy-driven jumps scaled by a seasonal coefficient. The seasonal value transitions from exploration to exploitation:

$$S_{\max} = 2 \left(1 - \frac{t}{T} \right), \quad S_{\min} = \frac{t}{2T}$$

$$S = S_{\min} + \xi (S_{\max} - S_{\min}), \quad \xi \sim U[0, 1] \quad (3)$$

The Lévy step is computed as:

$$\mathcal{L}^{(j)} = \frac{u^{(j)}}{|v^{(j)}|^{1/\beta}}, \quad \beta = 1.5, \quad u^{(j)}, v^{(j)} \sim \mathcal{N}(0, 1)$$

Genome update:

$$g_j \leftarrow g_j + 0.01 \cdot G_c \cdot S \cdot \mathcal{L}^{(j)} \cdot (g_{\text{best},j} - g_j) \quad (4)$$

4.2.3 Foraging Movement. Foraging squirrels switch behavior based on predator probability $P_{dp} = 0.1$. A uniform sample $x \sim U[0, 1]$ determines the regime. If $x < P_{dp}$ (predator detected):

$$g_j \leftarrow g_j + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, 0.04) \quad (5)$$

Otherwise:

$$g_j \leftarrow g_j + (g_{\text{best},j} - g_j) \cdot \xi_j, \quad \xi_j \sim \mathcal{N}(0, 0.25) \quad (6)$$

The first case models predator-induced perturbation (small displacement); the second drives convergence toward the best solution, where negative ξ_j values permit occasional divergence for exploration.

4.3 Fitness Evaluation

Fitness evaluation decodes each genome to a prompt, evaluates it on the dataset via LLM inference, and quantifies performance via a weighted combination of accuracy and class-balanced F1 score. Here, $k \in \{1, \dots, N\}$ indexes examples; y_k is the ground-truth label, $\hat{y}_k$ the predicted label; C is the class set with support n_c and weight $w_c = n_c/N$; and P_c, R_c are per-class precision/recall.

$$\text{Acc} = \#(\hat{y}_k = y_k) / N$$

$$F_1^{\text{weighted}} = \sum_{c \in C} w_c \cdot \frac{2P_c R_c}{P_c + R_c}, \quad w_c = \frac{n_c}{N}.$$

Algorithm 1: Squirrel Search Algorithm for Prompt Optimization

```

1: Input: Population size  $p$ , maximum iterations  $T$ , dataset with  $N$  samples
2: Input: Constants: gravity  $G_c = 1.9$ , predator probability  $P_{dp} = 0.1$ , genome dimensions  $d = 10$ 
3: Output: Best genome and corresponding optimized prompt
// Phase 1: Initialize Population
4: for  $i = 1$  to  $p$  do
5:   Generate random genome values
6:   Decode genome to prompt; evaluate fitness
7: end for
// Phase 2: Main Optimization Loop
8: for  $t = 1$  to  $T$  do
9:   Sort squirrels by fitness (lower is better)
10:  Assign top 15% as elite, middle 35% as exploratory, bottom 50% as foraging
11:  Record best genome
  /* Elite Movement */
12:  Compute seasonal decay  $W = 2 \times (1 - t/T)$ 
13:  for each elite squirrel except best do
14:    for each genome dimension  $j$  do
15:      Sample noise  $\sim U[0, 1]$ 
16:       $g_j \leftarrow g_j + G_c \cdot W \cdot (g_{\text{best},j} - g_j) \cdot \text{noise}$ 
17:    end for
18:    Clip genome to  $[0, \max_j]$ 
19:  end for
  /* Exploratory Movement (Lévy Flight) */
20:  for each exploratory squirrel do
21:    Compute seasonal  $S$  from  $S_{\min}$  and  $S_{\max}$ 
22:    for each dimension  $j$  do
23:      Sample Lévy step from heavy-tailed distribution
24:       $g_j \leftarrow g_j + 0.01 \cdot G_c \cdot S \cdot \text{Lévy} \cdot (g_{\text{best},j} - g_j)$ 
25:    end for
26:    Clip genome to  $[0, \max_j]$ 
27:  end for
  /* Foraging Movement */
28:  for each foraging squirrel do
29:    Sample  $x \sim U[0, 1]$ 
30:    if  $x < P_{dp}$  then ▷ Predator detected
31:      for each dimension  $j$  do
32:        Apply Gaussian noise  $\mathcal{N}(0, 0.2^2)$ 
33:      end for
34:    else
35:      for each dimension  $j$  do
36:        Sample large step  $\xi \sim \mathcal{N}(0, 0.5^2)$ 
37:         $g_j \leftarrow g_j + (g_{\text{best},j} - g_j) \cdot \xi$ 
38:      end for
39:    end if
40:    Clip genome to  $[0, \max_j]$ 
41:  end for
  /* Evaluate Population */
42:  for each squirrel do
43:    Decode genome to prompt
44:    Evaluate prompt on dataset ( $N$  queries)
45:  end for
46:  if any squirrel improves best fitness, update best genome
47: end for
// Phase 3: Return Result
48: Decode best genome to final prompt
49: return best genome, optimized prompt, best fitness

```

SSA minimizes the composite fitness objective:

$$f_i = \alpha(1 - \text{Acc}) + (1 - \alpha)(1 - F_1^{\text{weighted}}), \quad \alpha = 0.6$$

where α weights overall correctness against class-wise performance. Lower fitness indicates superior prompt quality.

4.4 Computational Complexity

Table 2 shows the time complexity analysis of our implementation. Let p population size, N dataset size, T iterations:

Table 2: SSA Computational Operations

Operation	Per iteration	Total
Sorting	$O(p \log p)$	$O(Tp \log p)$
Movement updates	$O(pd)$	$O(Tpd)$
Fitness evaluation	$O(pN)$	$O(TpN)$
Total LLM Calls	pN	TpN

$$O(T \cdot p \cdot N)$$

5 Experimental Evaluation

We evaluate SSA-optimized prompt tuning across three sentiment classification benchmarks (Table 3): the Stanford Sentiment Treebank (SST) [19], the Customer Review Dataset (CR) [7], and the Movie Review Dataset (MR) [14]. We employ three open-source LLMs of varying sizes: GPT-OSS:20b [13], Mistral:7b [9], and Phi-4-Mini:3.8b [12]. All datasets use a trinary label taxonomy (positive, negative, neutral). Experimental runs terminate early when no improvement is observed across 10 consecutive iterations.

Table 3: Sentiment Classification Datasets Used in the Experiment

Dataset	Labeled Instances
Stanford Sentiment Treebank	11,841
Customer Review Dataset	1,728
Movie Review Dataset	2,000

5.1 Hardware and Execution Configuration

All experiments were executed locally under the configuration summarized in Table 4. The models were invoked using Ollama with deterministic inference settings.

5.2 Baseline

We establish a baseline using the best-performing prompt from a randomly initialized population of 20 squirrels, each decoded from a uniformly random 10-dimensional genome vector using the same encoding scheme as SSA-optimized prompts. The initial fitness values reported in Table 5 (ranging from 0.0164 to 0.2120 across models and datasets) represent this random baseline, with no search or iterative refinement applied. This baseline provides a fair comparison by measuring the improvement gained through SSA

Table 4: System and Experimental Configuration

Component	Specification
<i>Hardware</i>	
CPU	13th Gen Intel Core i9-13900K
Memory	126 GB
GPU 0	NVIDIA RTX 4090 (24GB VRAM)
GPU 1	NVIDIA RTX 4090 (24GB VRAM)
<i>SSA Parameters</i>	
Population size	20 squirrels
Max iterations	50
Gravity constant (G_c)	1.9
Predator probability (P_{dp})	0.1
Early stopping	10 stagnant iterations
LLM Temperature	0.0

optimization, while isolating the contribution of the metaheuristic search process from other factors like prompt engineering expertise or manual template design.

6 Results

SSA convergence behavior and final accuracies for all evaluated models are reported in Table 5. In every experiment, squirrel populations converged before reaching the maximum number of iterations, and runs were terminated early after 10 consecutive iterations without fitness improvement.

6.1 Comparison with Baseline

To assess whether SSA’s improvements stem from effective optimization or merely fortunate initialization, we compare final results against the initial random population’s performance.

Across all experiments, SSA consistently outperformed random initialization. On the CR dataset, it achieved accuracies of 92% with GPT-OSS:20b, 96% with Mistral:7b, and 93% with Phi-4-Mini:3.8b, surpassing baseline accuracies of 91%, 89%, and 87%, respectively, and resulting in measurable absolute and relative gains.

However, GPT-OSS:20b’s 0% improvement on the MR dataset suggests either that the randomly initialized baseline was already well-optimized or that the model’s decision landscape constrained SSA’s search effectiveness; in contrast, Mistral:7b and Phi-4-Mini:3.8b demonstrated improvements on the same dataset (23.1% and 51.2%, respectively), indicating that model architecture and size significantly influences optimization outcomes.

6.2 Convergence Analysis

Across all evaluated models and datasets, the Squirrel Search Algorithm (SSA) consistently converged well before reaching the maximum iteration budget of 50. In most experimental runs, convergence was observed within 10–30 iterations.

Convergence rate varied as a function of model architecture. For instance, Mistral:7b converged notably faster on the SST dataset (14 iterations) compared to GPT-OSS:20b (27 iterations), suggesting that model architecture may influence optimization dynamics. The convergence iteration heatmap (Figure 1) for Phi-4-Mini:3.8b on

Table 5: SSA Performance: Fitness Optimization and Classification Accuracy

Dataset	Metric	GPT-OSS:20b				Mistral:7b				Phi-4-Mini:3.8b			
		Iter	Acc (%)	Fit. Init	Fit. Final	Iter	Acc (%)	Fit. Init	Fit. Final	Iter	Acc (%)	Fit. Init	Fit. Final
CR	Values	11	92	0.0893	0.0786	24	96	0.0840	0.0361	25	93	0.1226	0.0642
	Improvement			12.0%				57.0%				47.6%	
SST	Values	27	80	0.2120	0.1910	14	86	0.1831	0.1379	33	86	0.1825	0.1423
	Improvement			9.9%				24.7%				22.0%	
MR	Values	10	90	0.0945	0.0945	19	90	0.1229	0.0945	13	99	0.0164	0.00801
	Improvement			0%				23.1%				51.2%	

Iter: Convergence iterations, *Acc*: Final classification accuracy, *Fit. Init/Final*: Composite Fitness (baseline vs. optimized), *Improvement*: Relative fitness improvement, calculated as $\left| \frac{\text{Final} - \text{Init}}{\text{Init}} \right| \times 100$.

SST further corroborates this pattern, demonstrating that the most substantive genomic changes occur within the first 15 iterations.

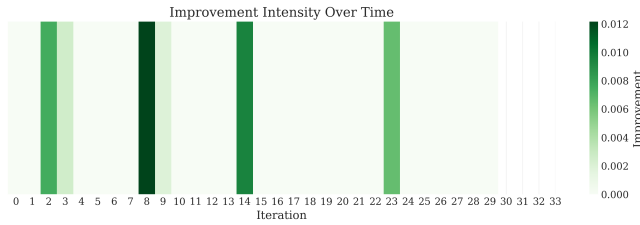


Figure 1: Improvement Heatmap for Phi-4-Mini on SST

The consistent reduction from initial to final fitness values across runs demonstrates steady improvement over the seeded population, indicating that the search process effectively refines prompt configurations beyond their initial states.

6.3 Population Dynamics

Population dynamics in SSA-optimized prompt tuning balance exploration and exploitation. Initially, high diversity enables broad search, while later iterations see reduced diversity as the population converges on high-quality solutions, avoiding premature convergence.

Figure 2 illustrates this behavior for Mistral:7b on the MR dataset. Over 19 iterations, the best fitness shows significant improvements during the initial 9 iterations, marked by sharp bursts of progress, followed by a gradual decline in diversity as the population converges. The most significant gains occur within the first four iterations, in the subsequent iterations, diversity continues to decrease as the population concentrates around optimal solutions, marking the transition to exploitation.

6.4 Emergent Prompt Patterns

Table 6 exhibits the transformation of prompts through SSA optimization on Mistral:7b applied to the Customer Review (CR) dataset. The initial randomly sampled prompt, through evolutionary refinement across 24 iterations, changes to, the optimized version which adopts an expert role persona, introduces implicit sentiment reasoning, and simplifies the output format to a single label.

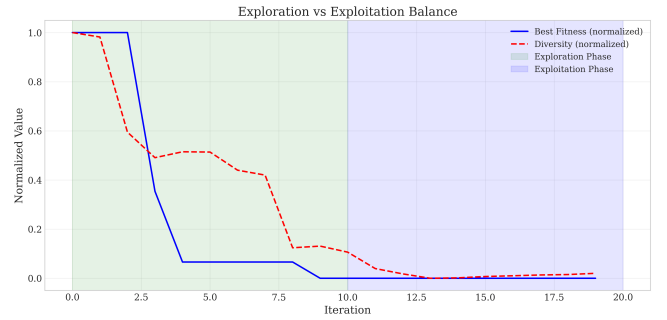


Figure 2: Fitness and Diversity Trajectory for Mistral:7B

Table 6: Prompt Evolution: Mistral:7b on CR (Fitness: 0.0840 → 0.0361, 24 iter.)

Stage	Prompt
Baseline	You are an advanced text sentiment classifier. Categorize the emotional stance expressed here. <i>Constraints</i> : Classify with high precision. Double-check your reasoning. <i>Format</i> : Classification Confidence Reasoning.
SSA Optimized	Act as a sentiment classification expert . Classify the sentiment of this material. Identify contrasts between stated and implied sentiment. <i>Constraints</i> : Classify with high precision. Double-check your reasoning. <i>Format</i> : Answer with just the sentiment label, nothing else.

It is worth noting that the baseline prompt shown in Table 6 represents the best-performing genome from the randomly initialized population, which accounts for its relative coherence. For comparison, the worst-performing squirrel in that same initial population produced the following prompt:

“You are a natural language understanding specialist. Evaluate the polarity of this input as favorable, disparaging, or neutral. Constraints: Be careful with your classification. Output format: Classify, then explain your reasoning in one sentence.”

The poor performance of this prompt can be attributed to context pollution from including the words “favorable,” “disparaging,” or “neutral,” which conflicted with the experimental requirement for “positive,” “negative,” or “neutral” labels, and the presence of vague constraints.

6.5 Computational Efficiency and Scalability

Table 7 reports wall-clock time, per-iteration cost, and token consumption across models on the CR dataset (1,728 samples, population size 20).

Table 7: Execution Time and Token Usage on the CR Dataset

Model	Iter	Time	Avg./Iter	Tokens	Avg./Iter
GPT-OSS:20b	11	4.20 hrs	25.18 min	4.58M	416K
Mistral:7b	24	0.69 hrs	2.94 min	6.27M	179K
Phi-4:3.8b	25	3.82 hrs	8.82 min	5.75M	221K

Wall-clock time and token consumption scaled differently across models (Table 7). Despite using 24% more total tokens, Mistral:7b achieved $6.1\times$ faster wall-clock time than GPT-OSS:20b due to lower per-token latency, confirming that inference latency, and not token count—dominates the optimization timeline. Token usage variation stems partly from prompt caching: repeated evaluations benefit from cache hits, reducing actual token costs below measured counts.

Scaling to larger corpora would increase token usage linearly, though cache hits could reduce the overhead. Nevertheless, total budgets remain tractable, making SSA feasible for practitioner adoption.

7 Conclusion

We presented a gradient-free framework for automated prompt optimization using the Squirrel Search Algorithm (SSA) [8]. By encoding prompts as continuous genomes mapped to discrete prompt components—instruction templates, reasoning strategies, role personas, and output formats. Our approach enables population-based metaheuristic search over the combinatorial prompt space without requiring gradient access or internal model parameters. Across three sentiment classification benchmarks and three open-source LLMs (3.8B–20B parameters) [9, 12, 13], SSA achieved relative fitness improvements of up to 57.0%, with all runs converging within 10–33 iterations. Beyond measurable fitness gains, SSA discovered emergent prompt strategies, including role specialization and simplified output formatting, demonstrating that structured evolutionary search can uncover effective prompt patterns without human intervention. These results establish nature-inspired metaheuristics as a viable and practical pathway for black-box LLM adaptation, particularly in API-restricted environments where gradient-based methods are infeasible.

8 Limitations and Future Work

This work has several limitations. Our evaluation compares SSA against random initialization but does not include direct comparisons with other gradient-free methods such as EvoPrompt [4] or GrIPS [15], partly due to differences in model access and implementation complexity. Experiments focus exclusively on sentiment classification across three datasets; evaluating SSA on diverse tasks (e.g., question answering, reasoning) and larger corpora would better establish generalizability. Additionally, our evaluation covers small-to-medium open-source models (3.8B–20B parameters), and results may differ for larger proprietary models or smaller architectures. Finally, SSA hyperparameters were adapted from prior

work [8] without systematic sensitivity analysis. Despite these limitations, our results demonstrate SSA’s viability for gradient-free prompt optimization in black-box settings.

References

- [1] Ilhem Boussaïd, Julien Lepagnot, and Patrick Siarry. 2013. A Survey on Optimization Metaheuristics. *Information Sciences* 237 (2013), 82–117. <https://doi.org/10.1016/j.ins.2013.02.041>
- [2] Angelica Chen, David M. Dohan, and David R. So. 2023. EvoPrompting: Language Models for Code-Level Neural Architecture Search. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., New Orleans, USA, Article 342, 31 pages.
- [3] Henning Titi Ciptaningtyas, Ary Mazharuddin Shiddiqi, Diana Purwitasari, Fuad Dary Rosyadi, and Muhammad Nur Fauzan. 2024. Multi-Objective Task Scheduling Algorithm in Cloud Computing Using Improved Squirrel Search Algorithm. *International Journal of Intelligent Engineering and Systems* 17, 1 (2024), 895–912. <https://doi.org/10.22266/ijies2024.0229.74>
- [4] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2023. Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. <https://doi.org/10.48550/arXiv.2309.08532> arXiv:2309.08532
- [5] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2021. Towards a Unified View of Parameter-Efficient Transfer Learning. arXiv:2110.04366 <https://arxiv.org/abs/2110.04366>
- [6] Huu-Tuong Ho, Thi-Thuy-Hoai Nguyen, Duong Nguyen Minh Huy, and Luong Vuong Nguyen. 2025. Bio-Inspired Algorithms in NLP Techniques: Challenges, Limitations and Its Applications. *Computers, Materials & Continua* 83, 3 (2025), 3945–3973. <https://doi.org/10.32604/cmc.2025.063099>
- [7] Minqing Hu and Bing Liu. 2004. Mining and Summarizing Customer Reviews. In *Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '04)*. Association for Computing Machinery, Seattle, USA, 168–177. <https://doi.org/10.1145/1014052.1014073>
- [8] Mohit Jain, Vijay Singh, and Anita Rani. 2019. A Novel Nature-Inspired Algorithm for Optimization: Squirrel Search Algorithm. *Swarm and Evolutionary Computation* 44 (2019), 148–175. <https://doi.org/10.1016/j.swevo.2018.02.013>
- [9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
- [10] James Kennedy and Russell Eberhart. 1995. Particle Swarm Optimization. In *Proceedings of ICNN'95-international conference on neural networks*, Vol. 4. IEEE, Perth, Australia, 1942–1948. <https://doi.org/10.1109/ICNN.1995.488968>
- [11] Salma Kharrat, Fares Fourati, and Marco Canini. 2025. ACING: Actor-Critic for Instruction Learning in Black-Box LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 19075–19102. <https://doi.org/10.18653/v1/2025.emnlp-main.965>
- [12] Microsoft. 2025. Phi-4-Mini Technical Report: Compact Yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv:2503.01743 [cs.CL] <https://arxiv.org/abs/2503.01743>
- [13] OpenAI. 2025. GPT-OSS-120B & GPT-OSS-20B Model Card. arXiv:2508.10925 [cs.CL] <https://arxiv.org/abs/2508.10925>
- [14] Bo Pang and Lillian Lee. 2004. A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts. *CoRR* cs.CL/0409058 (2004). <http://arxiv.org/abs/cs.CL/0409058>
- [15] Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. 2023. GrIPS: Gradient-Free, Edit-Based Instruction Search for Prompting Large Language Models. arXiv:2203.07281 [cs.CL] <https://arxiv.org/abs/2203.07281>
- [16] K.V. Price. 1996. Differential Evolution: A Fast and Simple Numerical Optimizer. In *Proceedings of North American Fuzzy Information Processing*. IEEE, Berkeley, USA, 524–527. <https://doi.org/10.1109/NAFIPS.1996.534790>
- [17] Colin Reeves. 2010. *Genetic Algorithms*. Vol. 146. Springer, New York, USA, 109–139. https://doi.org/10.1007/978-1-4419-1665-5_5
- [18] Antonio Sabbatella, Andrea Ponti, Ilaria Giordani, Antonio Candelieri, and Francesco Archetti. 2024. Prompt Optimization in Large Language Models. *Mathematics* 12 (2024), 929. <https://doi.org/10.3390/math12060929>
- [19] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Seattle, USA, 1631–1642.
- [20] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2025. Parameter-Efficient Fine-Tuning in Large Models: A Survey of Methodologies. arXiv:2410.19878 [cs.CL] <https://arxiv.org/abs/2410.19878>